Abhinand
Department of Mechanical Engineering
Indian Institute of Technology Madras

IIT Madras

October 21, 2026
Git Revision: d76603b

All results reproducible from committed artifacts

Outline

1. Introduction
2. Related Work
3. Digital Twin Physics
4. Environment Design
5. Reinforcement Learning
6. Results
7. Interactive Workstation
8. Software Engineering
9. Discussion
10. Conclusion

Problem: Dynamic Cutting Parameter Optimization

CNC face milling: rotating cutter removes material in multiple passes.

Parameters to optimize:

- Spindle speed n [RPM]
- Feed per revolution f_r [mm/r]
- Axial depth of cut a_p [mm]

Four competing objectives:

- **Productivity:** Maximize $\mathrm{MRR} = v_c \cdot a_p \cdot f_r$
- **Energy:** Minimize total electrical energy/part
- **Quality:** $R_a \leq 1.6$ μm (finished surface)
- **Tool Life:** $V_B \leq 0.3$ mm, avoid breakage

Traditional fixed parameters (handbooks, static NSGA-II) cannot adapt to tool wear, material variation, or geometry changes.

Proposed Approach

1. **Digital Twin (Physics)**
   - Speed-dependent cutting energy $k_c(v_c) = k_{c0}(v_c/v_{\text{ref}})^m$
   - Extended Taylor tool wear
   - Empirical roughness regression
   - Stochastic breakage
2. **DRL Agent (PPO)**
   - One Gymnasium environment
   - 11-D observation, context always present
   - Energy-aware reward
   - Conditioned policy (w_2 + material context)
3. **Experimental Rigor**
   - 100-scenario Monte Carlo + 95% CIs
   - Ablation: 7 weights $\times$ 3 seeds
   - 5 classical baselines + receding-horizon
   - Robustness: 3 regimes against the controller
   - Batch: one tool, ten parts
   - Transfer experiment
   - Full reproducibility
4. **Interactive Workstation**
   - 2D stock depletion, micro-topography
   - G-code generation, multi-controller comparison

Positioning Against Reference Papers

Capability	Mo et al. (2025) *J. Manuf. Process 156*	Wang et al. (2025a) *J. Manuf. Syst. 78*	**This Work**
Dynamic optimization	✓	✓	✓
Tool wear in the state	✓	✗	✓
Roughness constraint	✓	✗	✓
Energy as an objective	✗	✗	✓
Speed-dependent k_c	✗	✗	✓
Domain randomization	✗	✗	✓
Conditioned policy	✗	✗	✓
Transfer experiment	✗	✗	✓
Receding-horizon baseline	✗	✗	✓
Robustness vs a model-based controller	✗	✗	✓
Monte Carlo (100 scenarios)	✗	✗	✓
Interactive workstation	✗	✗	✓

Mo *et al.*: PPO over a mixed action space (tool change + parameters); the state carries V_B, breakage and R_a; objectives are quality and efficiency.
Wang *et al.*: MATD3 for batch milling; objectives are dimensional quality and machining time.

Key differentiator: **neither reference treats energy as an objective**. The speed-dependent k_c creates a genuine energy/throughput trade-off, and one conditioned policy covers the whole frontier in a single run.

Key Physics Equations (MillingDT/config/physics.py)

$$v_c = \frac{\pi D n}{1000} \tag{1}$$

$$\mathrm{MRR} = v_c \cdot a_p \cdot f_r \tag{2}$$

$$t_{\mathrm{cut}} = \frac{L_{\mathrm{cut}}}{n f_r} \tag{3}$$

$$\Delta V_B = C_{\mathrm{wear}} v_c^{\alpha} f_r^{\beta} a_p^{\gamma} t_{\mathrm{cut}} \tag{4}$$

$$R_a = k_1 f_r^2 + k_2 V_B + \frac{k_3}{v_c} \tag{5}$$

$$k_c(v_c) = k_{c0} \left(\frac{v_c}{v_{\mathrm{ref}}} \right)^m \tag{6}$$

$$P_{\mathrm{total}} = P_{\mathrm{idle}} + k_c(v_c) \cdot \mathrm{MRR} \tag{7}$$

$$E_{\mathrm{pass}} = P_{\mathrm{total}} \cdot t_{\mathrm{cut}} \tag{8}$$

Key Insight: Interior Minimum

With $m = 0.6 > 0$:

- Idle energy $\propto 1/(n f_r) \downarrow$ with speed
- Cutting energy $\propto n^m \uparrow$ with speed
- Part energy $E_{\mathrm{part}}(v_c)$ has **interior minimum**
- Energy weight w_2 selects operating point

Wear: $C_{\mathrm{wear}} = 1.0 \times 10^{-2}$, anchored on a published tool life (about 9 parts per tool): none of the three reference papers carries a Taylor wear model.

One Gymnasium Environment: `MillingDTEnv-v1`

Two registrations

- `env.ENV_ID` = `MillingDTEnv-v1`, 200-step limit
- `env.BATCH_ENV_ID` = `MillingDTBatchEnv-v1`
- The energy weight and the material are *inputs*, not separate tasks

Observation (11-D, always the same shape)

- 0–7: tanh-squashed V_B, κ_{rem}, previous n, f_r, a_p, $\mathrm{MRR}/\mathrm{MRR}_{\max}$, tanh-squashed R_a, tool-broken flag
- 8–10: the context: w_2, C_{wear} over $[0.6, 1.4]$, k_1 over $[0.8, 1.2]$ of nominal

Action: $[-1, 1]^3 \to$ clipped $\to$ machine envelope

Reward per step

$$r = 100\left(w_1 \frac{\mathrm{MRR}}{\mathrm{MRR}_{\max}} - w_2 \frac{E_{\text{pass}}}{E_{\text{ref}}}\right) + 20\,\Delta\left(\frac{\kappa_{\text{removed}}}{\kappa_{\text{total}}}\right) \tag{9}$$

$$- 50\,\mathbb{1}[V_B > V_{B,\text{safe}}] - 100\,\mathbb{1}[\text{broken}] \tag{10}$$

$$- 50\,\mathbb{1}[\text{finish} \wedge R_a > R_{a,\text{safe}}] - 0.05 \tag{11}$$

- $+200$ when a part is finished in spec, plus a Pareto margin bonus of at most $+5$
- $w_1 = 1 - w_2$; safety limits $V_{B,\text{safe}} = 0.255$ mm and $R_{a,\text{safe}} = 1.36$ μm
- breakage $p = 0.3 \cdot$ wear factor $\cdot$ (load factor)^2 above $V_B > 0.15$ mm
- the scale is a design decision: the objective term is worth about 50 points per part, a quarter of a part

Domain Randomization

- **Workpiece:** $\kappa_{\text{total}} \in [2, 15]$, $D \in [40, 150]$, $L_{\text{cut}} \in [50, 200]$
- **Material/Tool:** C_{wear} at $[0.6, 1.4]\times$ nominal, k_1, k_2, k_3 at $[0.8, 1.2]\times$ nominal
- **Initial wear:** $V_{B,0} \sim \mathcal{U}[0, 0.8]\, V_{B,\max}$
- **Energy weight:** drawn $\mathcal{U}[0, 1]$ per episode under `-randomize-weight`
- **Shared tables:** same `MillingDT/config/scenarios.py` for training + Monte Carlo eval
-
- **Measured:** the normalized physics features the policy sees on the training and the evaluation sets agree to 0.041 in the mean (std = 1), the same distribution
- **In the observation:** C_{wear} and k_1 travel with the part, so the policy knows which material it is cutting
- **Belief and noise:** the context can carry a wrong material and the two measured features can carry noise, which is what the robustness experiment uses

PPO Training Setup (Stable-Baselines3)

Table: Hyperparameters

Parameter	Value
Policy	MlpPolicy (2×256)
Learning rate	3×10^{-4} (linear)
n_steps	2048
Batch size	64
n_epochs	10
γ / GAE λ	0.99 / 0.95
Clip range	0.2
Entropy coeff	0.02
VecNormalize	norm_obs=True, norm_reward=True
Envs / Device	4 (`SubprocVecEnv`) / CPU

Training Runs

- **Production policy:** 1M steps, seed 42, `-randomize -randomize-weight` → `results/run_PPO_seed42/`
- **Ablation grid:** 7 w_2 × 3 seeds = 21 policies, 150k steps each
- **Batch-aware policy:** 1M steps on `MillingDTBatchEnv-v1`; **noisy-measurement policy:** 1M steps with `-obs-noise 0.05`

Every ablation policy has its weight pinned at training and evaluated at that same weight; the production policy sees the weight drawn per episode.

Heartbeat callback every 1000 steps → stall detection

Production Policy: Monte Carlo Sweep (100 Scenarios)

Metric	Mean	Std	95% CI
MRR (mm^3/min)	1608.5	550.7	[1499.2, 1717.8]
Energy (kJ)	66.4	48.0	[56.9, 75.9]
Cycle time (s)	11.4	6.9	[10.1, 12.8]
Max R_a (μm)	1.610	0.178	[1.575, 1.645]
Max V_B (mm)	0.1407	0.0672	[0.1274, 0.1540]

Constraint	Rate	95% CI
R_a viol., worst pass	55.0%	[45.2, 64.8]%
R_a viol., finished surface	4.0%	[0.2, 7.8]%
R_a above safety limit, worst	87.0%	[80.4, 93.6]%
R_a above safety limit, finished	4.0%	[0.2, 7.8]%
$V_B > 0.3$ mm	0.0%	[0, 0]%
Tool breakage	6.0%	[1.3, 10.7]%
Parts completed in spec	94.0%	

Key Findings

- A much more aggressive operator than under the old reward: 1608.5 mm^3/min against 1205.2, and 11.4 s per part against 20.3 s
- The price is paid in the tool and in the part quality: 6.0% tool breakage and 4.0% of the finished surfaces outside the roughness limit, 87% of the worst passes above the safety margin
- Wear is not the binding constraint on a single part (0% violations, mean 0.1407 mm); it binds on the aggregate of a batch
- Mean operating point: 7907 RPM, 0.318 mm/r, 2.07 mm, 4.0 passes

Energy Weight Ablation: Empirical Pareto Frontier

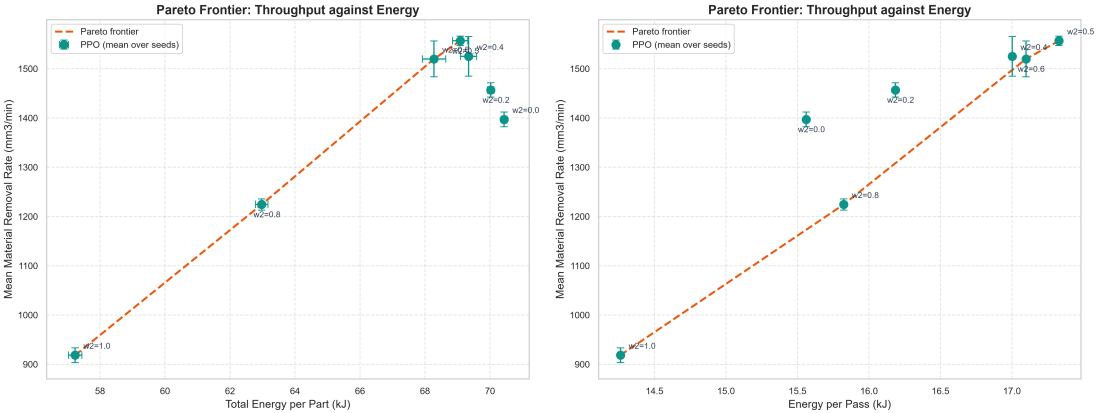
Table: Ablation: mean ± std over 3 seeds, 50 scenarios each, weight pinned

w_2	MRR	Energy (kJ)	Cycle (s)	n (RPM)	f_r	Frontier
0.0	1397.1±14.8	70.4±0.1	14.8	7998	0.299	dominated
0.2	1456.9±14.6	70.0±0.1	13.8	7992	0.299	dominated
0.4	1524.9±40.2	69.3±0.2	13.0	7896	0.296	dominated
0.5	1556.6±9.7	69.1±0.2	12.7	7859	0.296	✓
0.6	1519.8±36.4	68.3±0.4	13.1	7638	0.296	✓
0.8	1224.4±11.3	63.0±0.2	15.4	6573	0.292	✓
1.0	918.5±14.9	57.2±0.2	20.1	5382	0.292	✓

Frontier Analysis

From $w_2 = 0.5$ to $w_2 = 1.0$ the removal rate falls 41% (1557 → 919 mm^3/min) and the energy falls 17% (69.1 → 57.2 kJ). The trade-off is **asymmetric**: the energy is bought with a lot of removal rate, and with time (12.7 → 20.1 s). The spindle speed falls with the energy weight (7859 → 5382 RPM). Dominated points ($w_2 = 0.0, 0.2, 0.4$) remove less for more.

Ablation Frontier: Removal Rate against Energy



Seven pinned-weight policies, three seeds each. The four non-dominated points are joined into the frontier: $w_2 = 0.5$, 0.6, 0.8, 1.0, and the three dominated points sit off it. Left panel: the energy of a part. Right panel: the energy of a pass, which the reward charges.

One Policy Against 21 Policies

w_2	MRR, one policy	MRR, per weight	MRR gap	Energy, one policy	Energy, per weight	Energy gap
0.0	1475.9	1397.1	+5.6%	68.4	70.4	−2.9%
0.2	1518.0	1456.9	+4.2%	65.8	70.0	−6.1%
0.4	1608.5	1524.9	+5.5%	66.4	69.3	−4.2%
0.5	1630.3	1556.6	+4.7%	66.0	69.1	−4.4%
0.6	1540.8	1519.8	+1.4%	65.5	68.3	−4.0%
0.8	1114.5	1224.4	−9.0%	57.9	63.0	−8.0%
1.0	720.4	918.5	−21.6%	51.7	57.2	−9.7%

Result (the claim of the previous revision has reversed)

Over the weights from 0.0 to 0.6 the single policy removes 1.4% to 5.6% more material for 2.9% to 6.1% less energy. At $w_2 = 0.8$ and $w_2 = 1.0$ it is behind by 9.0% and 21.6% of the removal rate. The weight in the observation is a weaker signal than a dedicated policy where the trade-off is steepest.

The same policy swept at runtime: MRR 1475.9 ($w_2 = 0.0$) → 720.4 ($w_2 = 1.0$), energy 68.4 → 51.7 kJ, spindle 8000 → 4111 RPM. **The weight stays a runtime query**, not a retrain, but it is not free at the ends of the range.

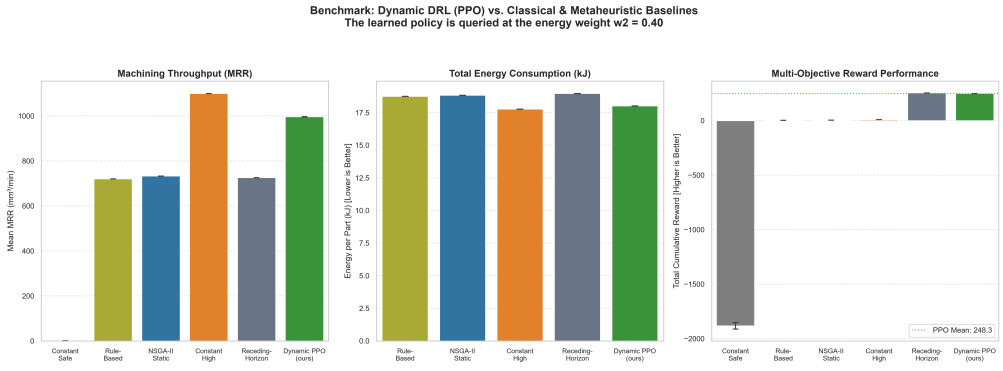
Baseline Comparison (50 Episodes, Nominal Condition, $w_2 = 0.4$)

Table: Baseline comparison on the same 50 episodes

Policy	Reward	MRR	Energy (kJ)	Cycle (s)	R_a final (μm)
Constant high	10.0	**1099.6**	17.8	**4.3**	1.56 (violates)
NSGA-II static	5.1	732.9	18.8	6.4	1.56 (violates)
Rule based	3.3	719.8	18.8	6.6	1.56 (violates)
Receding-horizon	253.5	725.5	19.0	6.7	1.25
Constant safe	-1882.9	1.6	385.7	768.0	0.26

- **Same 50 episodes** of the nominal condition for all six policies, at $w_2 = 0.4$
- **Three of the four static policies finish outside the roughness safety limit** (1.56 μm against 1.36 μm): no completion bonus, the finishing penalty, and they land near zero
- Constant-Safe stays inside the limit (0.26 μm) but needs 768 s and 385.7 kJ per part: -1882.9
- **Receding-horizon** is the strongest baseline, $+253.5$, and the only classical policy that finishes every part in spec; it pays with throughput: 725.5 mm^3/min and 6.7 s

Policy Comparison: Six Policies, Same 50 Episodes



The three fast static policies land near zero: they finish outside the roughness safety limit, so they are paid no completion bonus and are charged the finishing penalty. Constant-Safe stays inside but needs 768 s per part. The comparison that matters is the receding-horizon controller against the learned policy, the only two with a positive reward.

Robustness Against the Classical Controller

Regime	Arm	Reward	Std	MRR	kJ	s	Broken	Done
Controller has every advantage	Receding-horizon	248.9	79.5	1318.2	68.9	17.1	3%	96%
	PPO	233.6	79.7	1608.5	66.4	11.4	6%	94%
Material model is wrong	Receding-horizon	63.5	117.1	1306.9	71.0	23.3	2%	95%
	PPO	226.9	88.3	1604.2	66.3	11.5	6%	94%
Wear measurement is noisy	Receding-horizon	181.3	185.9	1300.4	74.2	31.1	4%	91%
	PPO trained with noise	213.6	87.3	1554.2	66.9	12.0	6%	94%

The same 100 randomized scenarios at $w_2 = 0.4$ in every row. The regime changes what the controller is given, not what the policy is given: the exact model, the exact state and the material in its hand in the first regime, a material it has no model for in the second, a 5% noise on the wear and the remaining stock in the third.

Robustness: How to Read It

1. **Inside a perfect digital twin the two methods are comparable, and the measured gap is partly an artifact of the reward.** The controller computes the optimum of the same objective at every pass with the exact model and the exact state, and it scores 248.9 against the policy's 233.6. The gap is inside the noise: the standard error of the mean is about 8 for both arms, so a difference of 15 has a t of about 1.4. And about nine of those fifteen points are the reward: the controller cuts the part in 5.85 passes against the policy's 4.00 and is paid for every pass.
2. **The learned policy is the better throughput operator.** It removes 22% more material per minute (1608 against 1318), finishes a part in a third less time (11.4 against 17.1 s) and spends 3.7% less energy. It pays with twice the tool breakage (6% against 3%).
3. **The advantage appears where a plant actually is.** With a material the controller has no model for, its reward collapses by three quarters, from 248.9 to 63.5, and the policy holds at 226.9. With a noisy wear measurement the controller falls to 181.3 and its spread doubles to 185.9 against the policy's 87.3, and the policy trained with the same noise keeps 213.6.
4. **The policy does not need the material to be told.** Told the truth it scores 215.1, against 226.9 for the policy that believes the nominal material.

The claim

Inside a perfect digital twin the learned policy and the model-based controller are comparable: the policy is ahead on the physical metrics and behind on the breakage, and the gap the reward does show is partly the artifact of how the reward pays for a rate. The advantage of the learned policy appears where the controller's information advantage disappears. The sensitivity of the comparison to the reward is published with the result.

Reward Sensitivity: How the Reward Pays for a Rate

The reward pays the removal rate of every pass, the standard way to express a rate in a reward that is paid per pass. The total credit therefore grows with the number of passes a plan takes, so a plan that cuts the same part in more, shallower passes is paid more for it.

On the same 100 randomized scenarios at $w_2 = 0.4$, measured by `experiments/reward_sensitivity.py`: the controller takes 5.85 passes against the policy's 4.00 and collects 27% more objective credit (+0.413 against +0.326) for the same material.

	Shipped form (sum of the per-pass rates)			Alternative form (material against energy)		
Objective scale	**Controller**	**Policy**	**Gap**	**Controller**	**Policy**	**Gap**
5	209.7	202.6	+7.1	208.2	201.6	+6.5
20	215.9	207.5	+8.3	209.7	203.4	+6.3
50	228.3	217.3	+10.9	212.9	207.1	+5.8
100	248.9	233.6	+15.3	218.2	213.2	+5.0
200	290.1	266.2	+23.9	228.7	225.3	+3.4
400	372.7	331.5	+41.2	249.9	249.6	+0.2
1000	620.2	527.2	+93.0	313.2	322.6	−9.4
4000	1857.9	1506.0	+352.0	630.1	687.4	−57.4

Mean reward of each arm on the same 100 scenarios, under two objective forms and eight objective scales. The gap is the controller's mean reward minus the policy's.

Reward Sensitivity: Three Checks and the Consequence

1. **The objective scale is not the lever**: the controller leads at every scale from 5 to 4000 and the gap only grows with it, 7.1 points at 5 and 352.0 at 4000, so the sign never flips.
2. **The part level alternative was implemented and rejected**: it removes the pass count from the accounting, so the arms are much closer (5.0 points at the scale of 100, against 15.3) and the policy is ahead at the large scales (9.4 points at 1000, 57.4 at 4000). But the material a part gives up is fixed by its stock, so the throughput weight would be inert and the weight sweep would stop measuring what it claims to measure.
3. **What a correct form would need**: a rate cannot be accumulated over a part without becoming a volume, so a reward has to either price the cycle time directly or compare the part's average rate to a reference at the end of the part. Both change the meaning of the weights, and the second makes the objective sparse and the credit assignment poor.

The consequence

The ideal case reads as a tie, not a win: the policy is ahead on the physical metrics (22% more material removed, a third less cycle time, 3.7% less energy) and behind on the tool breakage (6% against 3%), and the gap the reward does show is partly the reward's artifact. The reward as shipped carries it, documented rather than hidden.

Batch Against the Part

Arm	Batch reward	Std	Reward per part	Tool changes	Batch time (s)	Batch energy (kJ)
PPO, batch aware	1817.0	477.8	166.8	0.63	113.1	610.4
PPO, per part	1709.9	366.1	169.4	0.83	114.7	674.3
Receding-horizon, per part	1859.9	328.8	173.7	0.83	207.5	692.0

- A batch is ten parts cut with one tool; the tool is replaced when it breaks or reaches its wear limit, and the replacement costs 500 reward units against the 200 a finished part is worth. The observation carries the tool life left and the fraction of the batch left.
- The batch context is worth 24% fewer tool changes and 9.5% less energy at the same completion: the policy that sees the tool life and the batch that is left replaces the tool 0.63 times per batch against 0.83, and uses 610.4 kJ against 674.3 kJ. It also collects more batch reward, 1817.0 against 1709.9.
- The per-part controller gets the highest batch reward, 1859.9, and buys it with the machine time: 207.5 s for the batch against the batch-aware policy's 113.1 s, about 1.8 times the machine time for a batch reward that is 2.4% higher. It solves one part at a time, so it cannot price a tool change against the parts that follow it.

Controller Initialization: Method and Result

Question: does a policy that starts from the controller end up better than the expert it started from?

Method

Record the controller on 2000 randomized episodes → fit a supervised actor (2×256, Tanh) on those trajectories by regression → copy the fitted weights into a fresh PPO policy → train with the production budget (1M steps) and evaluate on 100 scenarios at $w_2 = 0.4$. `MillingDT/agent/controller_init.py` `experiments/controller_experiment.py`

Metric	Receding-horizon controller	PPO from scratch	PPO initialized from controller
Mean episode reward	248.9	233.6	119.8
Reward std	79.5	79.7	178.7
MRR (mm^3/min)	1318.2	1608.5	1274.9
Energy per part (kJ)	68.9	66.4	81.5
Cycle time (s)	17.1	11.4	42.6
R_a viol., finished	2%	4%	4%
Flank wear violations	1%	0%	9%
Tool breakage	3%	6%	4%
Parts completed	96%	94%	87%

Controller Initialization: What It Shows

The initialization is now harmful

- The initialized policy scores 119.8 against the from-scratch policy's 233.6, a gap of 113.8, more than fourteen times the standard error of the mean
- The clone starts the policy at the controller's conservative operating point, and with the wear constraint binding that conservatism is expensive
- It cuts a part in 42.6 s against 17.1 s and 11.4 s, and spends 81.5 kJ against 66.4 kJ
- It is the only arm that exceeds the flank wear limit (9%)

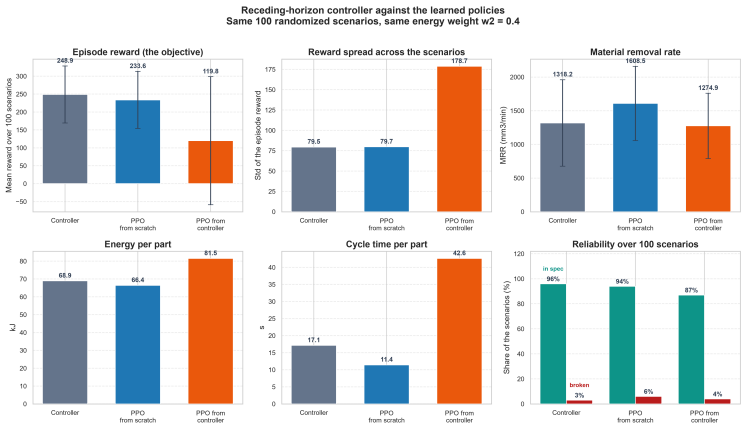
The result reversed

- The earlier revision found the initialization harmless: it made the policy more consistent and did not change the reward
- That was the reward not counting the cost of the slow operating point
- With the throughput counted, the initialization is what the measurement says it is
- What stays is the record and clone machinery, not the initialization itself

What the work claims

Inside a perfect digital twin the learned policy and the model-based controller are comparable: the policy is ahead on the physical metrics and behind on the breakage, and the gap the reward does show is partly the reward's artifact. Its advantage appears where the controller's information advantage disappears.

Controller Initialization: The Three Arms



Six panels, too detailed to read at slide size: the values are in the table. The initialized arm sits 114 points below the from-scratch arm on the reward, and its spread of 178.7 is more than twice the spread of the other two: the initialization does not buy consistency any more, it buys variance.

Transfer Experiment: Cross-Material Adaptation

Target material (harder alloy, same machine): $k_1 \times 1.25$, $k_2 \times 1.30$, $C_{\text{wear}} \times 2.0$. The source policy was trained for 300,000 steps on the nominal material; the adaptation budget on the target is 50,000 steps.

Arm	**Reward**	R_a **viol.** worst pass	R_a **viol.** finished surface	**Parts** completed
Source policy, zero shot	168.5	100.0%	6.7%	86.7%
Source policy, fine tuned (50k)	101.9	26.7%	0.0%	86.7%
Trained on the target from scratch (50k)	101.9	26.7%	0.0%	86.7%
Production policy, zero shot	190.1	100.0%	10.0%	90.0%

How to read this

The production policy was trained across a **distribution of materials** with the material context in its observation, so the harder material is inside its training distribution and it needs no adaptation at all: it scores the highest reward (190.1) and completes 90.0% of the parts. Its worst pass violates the roughness limit in every scenario, and its finished surface in 10.0% of them.

Adaptation repairs the roughness and costs more than it gains: fine tuning the nominal-trained policy for 50,000 steps, or training on the target from scratch, both cut the worst-pass violations to a quarter and land at 101.9 against the source policy's 168.5 zero shot. The target alloy wears the tool twice as fast, and a policy that slows down for that trades away more throughput than it gains in specification.

Streamlit Digital Twin Workstation

- **2D Stock Depletion:** Cylindrical billet cross-section; pass-by-pass cutout; remaining κ, a_p; tool engagement
- **Micro-Topography:** Feed-mark cusp $y_{\text{cusp}}(x) \approx x^2/(8r_\epsilon) + \text{noise}$; ISO 1302 R_a overlay; 1.6 μm ceiling
- **G-Code Generator:** ISO 6983 Fanuc/Siemens compliant; exact PPO sequence; tool/diameter aware
- **Multi-Controller:** Side-by-side PPO vs NSGA-II vs Rule-Based vs Constant-High; live telemetry
- **Cached Loading:** `@st.cache_resource` loads model once per session

```
streamlit run app.py
```

Architecture & Reproducibility

Module Structure

- `MillingDT/`: config, env (part + batch), agent (controller init), baselines, sim, utils
- `experiments/`: train, eval, ablation, baselines, transfer, controller, robustness, batch, reward sensitivity, pipeline
- `viz/`: shared plotting utilities
- `results/`: one directory per run

Reproducibility

- Global seeding (Python, NumPy, PyTorch, CUDA)
- Shared scenario tables (train = eval)
- `experiments/summarize_results.py` regenerates `RESULTS.md`
- No unused imports (AST verified)

Test Coverage (45 tests)

- 24 env tests: clipping, truncation, soft norm, 11-D observation, constants, overrides, energy norm, Gym checker, randomization, scenario scales, context (weight, material, belief), observation noise, energy minimum, reward golden values
- 11 batch env tests: the 13-D observation, the tool life and the batch left, the wear carried over, the tool change, the batch reward, the seeded reproducibility
- 5 integration, 3 callback, 2 training: roundtrip, registration, context, heartbeat, convergence

Discussion: What Worked

What Worked

- Speed-dependent k_c creates a genuine trade-off
- The scale of the reward decides the operating point: 50 points per part against 200 for a part
- Domain randomization $\rightarrow$ 94% completion
- One policy covers the range, ahead over $w_2 = 0.0$ to 0.6
- Training across materials makes the adaptation unnecessary

What Worked (continued)

- A receding-horizon controller that prices the whole reward is a fair and strong baseline ($184.3 \rightarrow 248.9$)
- Robustness: comparable inside a perfect twin, and the advantage appears where the controller's information runs out
- Measuring the reward found the artifact a fair-looking comparison would have hidden: the credit grows with the number of passes
- Batch context: 24% fewer tool changes, 9.5% less energy

Discussion: Limitations and Future Work

Limitations

- Inside a perfect twin the two methods are comparable: 248.9 against 233.6, the gap is inside the noise, and about nine of those points are the reward's artifact
- The learned policy is the better throughput operator, at twice the tool breakage
- Initializing from the controller is now harmful: 119.8 against 233.6
- The reward pays a rate per pass, so the credit grows with the number of passes: the bias is in the reward, and the scale is not the lever
- The wear tool life varies by only a factor of a few across the envelope
- Ablation: 150k steps per weight, so the frontier is a lower bound; sim-only, no real machine calibration yet

Future Work

- A reward that pays a rate correctly: price the cycle time directly, or compare the part's average rate to a reference at the end of the part
- Publish the sensitivity of any comparison to the reward form, as it is here
- Compare against a restricted controller (is the gap an information gap?)
- Sim-to-real calibration, true multi-objective RL, online w_2 adaptation, batch-level planning

Conclusion

- **Digital Twin** with speed-dependent cutting energy, tool wear, roughness, breakage; the wear coefficient is anchored on a published tool life
- **Two Gymnasium Environments** (the part, 11-D, and the batch, 13-D) and **One Production Policy** (1M steps): the energy weight is a runtime query, not a retrain
- **Rigorous Evaluation**: Monte Carlo (100 scenarios, 95% CIs), ablation frontier, weight sweep against the 21 per-weight policies
- **Baselines**: five classical policies on the same 50 episodes; the receding-horizon controller is the strongest at +253.5
- **Robustness**: inside a perfect twin the two methods are comparable (248.9 against 233.6, the gap inside the noise and partly the reward's artifact); the controller collapses to 63.5 and 181.3, the policy holds at 226.9 and 213.6
- **Reward sensitivity**: the reward pays the rate of every pass, so the credit grows with the pass count; the controller collects 27% more credit for the same material
- **Batch**: the batch context is worth 24% fewer tool changes and 9.5% less energy; the per-part controller needs 207.5 s against 113.1 s
- **Controller Initialization**: the result reversed, the initialization is now harmful (119.8 against 233.6)
- **Transfer Experiment**: the production policy needs no adaptation; the nominal-trained one does
- **Interactive Workstation** and **full reproducibility**: every result table from the committed artifacts by one script

Key Scientific Finding

The speed-dependent specific cutting energy creates a **genuine energy/throughput trade-off**, and DRL learns to navigate it pass-by-pass. Across the frontier the removal rate falls 41% while the energy of the part falls 17%. Against a model-based controller the claim is narrow: a tie, not a win.

Key Commands

Task	Command
Tests	`pytest tests/ -v`
Production	`experiments.train_rl -steps 1000000 -randomize -randomize-weight`
Pinned weight	`experiments.train_rl -steps 150000 -randomize -w2 0.6`
Noisy measurement	`experiments.train_rl -steps 1000000 -obs-noise 0.05`
Batch policy	`experiments.train_rl -env-id MillingDTBatchEnv-v1 -steps 1000000`
Ablation grid	`experiments.train_rl_ablation -steps 150000 -slots 7`
MC eval	`experiments.evaluate -scenarios 100`
Weight sweep	`experiments.evaluate -sweep-weight -scenarios 100`
Compare	`experiments.evaluate -compare-only`
Baselines	`experiments.run_baselines`
Transfer	`experiments.transfer_experiment 300000 50000`
Controller init	`experiments.controller_experiment 1000000 2000`
Robustness	`experiments.robustness_experiment`
Batch	`experiments.batch_experiment`
Reward sensitivity	`experiments.reward_sensitivity`
Results	`experiments.summarize_results`
Workstation	`streamlit run app.py`
Full pipeline	`experiments.run_pipeline -steps 1000000 -ablation_steps 150000`

Each entry is run as `python -m <module>` from the project root.

Thank You

Questions & Discussion

Repository: `G:/My Drive/IITM/AI in Mfg`
ME6324 – AI in Manufacturing | IIT Madras